



Module 3. Generative Adversarial Networks

Adversarial Training
Types of GANs
Applications of GANs

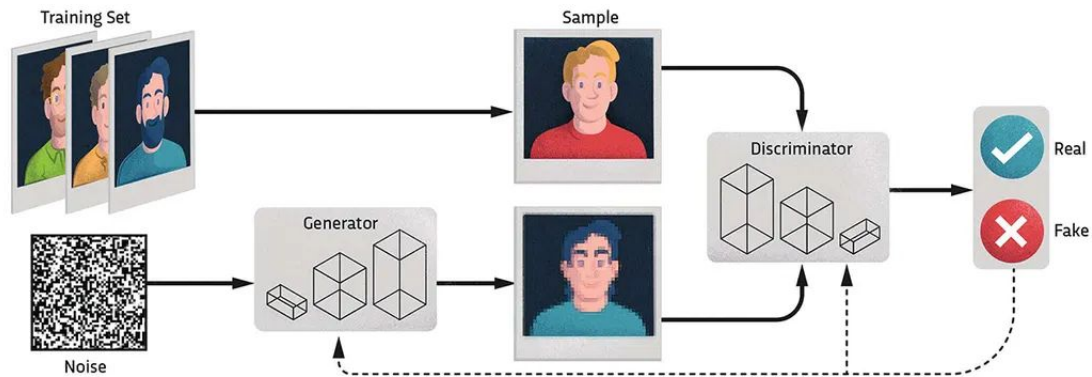


Adversarial Training

What are GANs?

- Introduced by Ian Goodfellow in 2014.
- Consist of **two neural networks**: the Generator and the Discriminator.
- Operate in a minimax game: the generator creates data to fool the discriminator, while the discriminator tries to distinguish between real and generated data.
- Widely used for generating realistic images, audio, text, and more.

- Generator (G): Produces synthetic data from random noise.
- Discriminator (D): Evaluates data samples, outputting a probability of authenticity.
- The two networks are trained simultaneously.
- The game is formulated as a minimax optimization problem.





Loss Function - The Generator Model

- The generator is a **deep neural network** that takes random noise as input to generate realistic data samples (e.g., images or text). It learns the underlying data distribution by adjusting its parameters through **backpropagation**.
- The generator's objective is to produce samples that the discriminator classifies as real.
- The loss function is:

$$J_G = -\frac{1}{n} \sum_{i=1}^n \log D(G(z_i))$$

- Where, J_G measure how well the generator is fooling the discriminator.
- $\log D(G(z_i))$ represents log probability of the discriminator being correct for generated samples.
- The generator aims to minimize this loss, encouraging the production of samples that the discriminator classifies as real $\log D(G(z_i))$, close to 1.

If $D(G(z)) = 0.9$: $\log(0.9) \approx -0.105 \rightarrow -\log(0.9) \approx 0.105$ (small loss).

If $D(G(z)) = 0.1$: $\log(0.1) \approx -2.303 \rightarrow -\log(0.1) \approx 2.303$ (big loss).

Minimization therefore encourages $D(G(z)) \rightarrow 1$.

Loss Function - The Discriminator Model

- The discriminator acts as a **binary classifier**, distinguishing between real and generated data. It learns to improve its classification ability through training, refining its parameters to detect fake samples more accurately.
- When dealing with image data, the discriminator often employs **convolutional layers or other relevant architectures suited to the data type**. These layers help extract features and enhance the model's ability to differentiate between real and generated samples.
- The discriminator reduces the negative log likelihood of correctly classifying both produced and real samples.



Loss Function - The Discriminator Model

- This loss incentivizes the discriminator to accurately categorize generated samples as fake and real samples with the following equation:

$$J_D = -\frac{1}{m} \sum_{i=1}^m \log D(x_i) - \frac{1}{m} \sum_{i=1}^m \log(1 - D(G(z_i)))$$

- Where, J_D assesses the discriminator's ability to discern between produced and actual samples.
- The log likelihood that the discriminator will accurately categorize real data is represented by $\log D(x_i)$
- The log chance that the discriminator would correctly categorize generated samples as fake is represented by $\log(1 - D(G(z_i)))$
- By minimizing this loss, the discriminator becomes more effective at distinguishing between real and generated samples.
 - If $D(x_i) = 0.9$: $\log(0.9) \approx -0.105 \rightarrow$ small penalty (good).
 - If $D(x_i) = 0.1$: $\log(0.1) \approx -2.303 \rightarrow$ large penalty (bad).
 - If $D(G(z_i)) = 0.1$: $\log(1 - 0.1) = \log(0.9) \approx -0.105 \rightarrow$ small penalty (good).
 - If $D(G(z_i)) = 0.9$: $\log(1 - 0.9) = \log(0.1) \approx -2.303 \rightarrow$ large penalty (bad).

MinMax Loss

- GANs follow a minimax optimization where the generator and discriminator are adversaries:

$$\min_G \max_D (G, D) = [\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(g(z)))]$$

- Where, G is generator network and D is the discriminator network
- Actual data samples obtained from the true data distribution $p_{data}(x)$ are represented by x.
- Random noise sampled from a previous distribution $p_z(z)$ (usually a normal or uniform distribution) is represented by z.
- $D(x)$ represents the discriminator's likelihood of correctly identifying actual data as real.
- $D(G(z))$ is the likelihood that the discriminator will identify generated data coming from the generator as authentic.

The generator aims to minimize the loss, while the discriminator tries to maximize its classification accuracy.

Training in Practice

- **Generator's First Move**

- G takes a random noise vector as input. This noise vector contains random values and acts as the starting point for G's creation process. Using its internal layers and learned patterns, G transforms the noise vector into a new data sample, like a generated image.

- **Discriminator's Turn**

- D receives two kinds of inputs:
 - Real data samples from the training dataset.
 - The data samples generated by G in the previous step.
- D's job is to analyze each input and determine whether it's real data or something G cooked up. It outputs a probability score between 0 and 1. A score of 1 indicates the data is likely real, and 0 suggests it's fake.

Training in Practice

- **Adversarial Learning**

- If the discriminator correctly classifies real data as real and fake data as fake, it strengthens its ability slightly. If the generator successfully fools the discriminator, it receives a positive update, while the discriminator is penalized.

- **Generator's Improvement**

- Every time the discriminator misclassifies fake data as real, the generator learns and improves. Over multiple iterations, the generator produces more convincing synthetic samples.

- **Discriminator's Adaptation**

- The discriminator continuously refines its ability to distinguish real from fake data. This ongoing duel between the generator and discriminator enhances the overall model's learning process.

Training in Practice

- **Training Progression**
 - As training continues, the generator becomes highly proficient at producing realistic data.
 - Eventually, the discriminator struggles to distinguish real from fake, indicating that the GAN has reached a well-trained state.
 - At this point, the generator can be used to generate high-quality synthetic data for various applications.

Challenges

- GAN training can suffer from issues like **mode collapse** (limited diversity in outputs) and **vanishing gradients** (insufficient learning signal), leading to instability.
- **Mitigation Strategies:** Techniques such as feature matching, mini-batch discrimination, and the adoption of alternative loss functions (like the Wasserstein loss) help alleviate these problems.
- **Architectural and Hyperparameter Tuning:** Choosing the right network architectures (for instance, using deep convolutional layers) and carefully tuning hyperparameters are essential for successful GAN training.

Alternative Loss Functions

- Wasserstein Loss: In Wasserstein GANs (WGANs)**, the discriminator (often called the critic) is trained to **approximate the Wasserstein distance between the real and generated data distributions**. This change leads to smoother gradients, which benefits the generator as well during its update step. In other words, while the critic's loss function is redefined, the improved gradients directly impact the generator's ability to learn better.
- Least Squares GAN (LSGAN)**: LSGAN replaces the standard cross-entropy loss with a least squares loss for both the generator and the discriminator. The idea is to minimize the Pearson χ^2 divergence, which helps alleviate issues like vanishing gradients by penalizing outputs based on their **distance from the target label**. This modification affects both networks' training objectives, leading to a more stable training process.

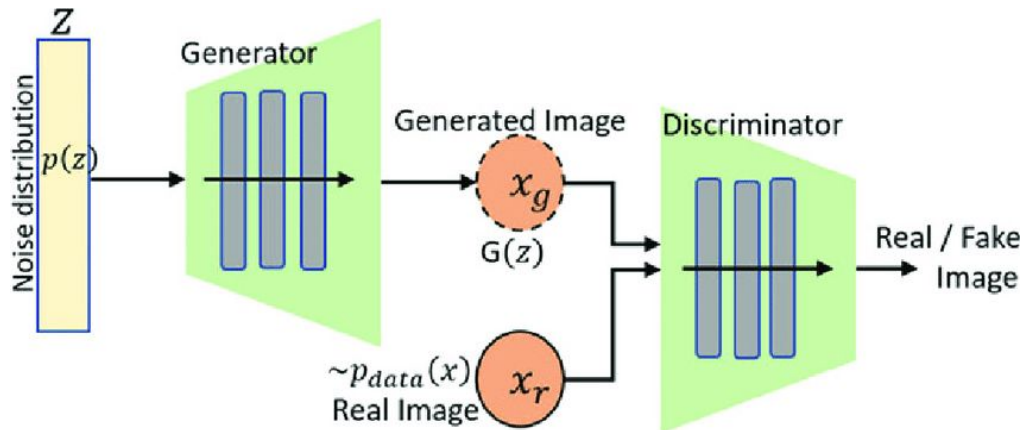
A series of white, stylized lines on the left side of the slide, resembling a circuit board or a series of parallel lines that curve and extend horizontally.

Types of GANs

Vainilla GANs

- Vanilla GAN is the simplest type of GAN. It consists of:
 - A generator and a discriminator, **both are built using multi-layer perceptrons (MLPs)**
 - The model optimizes its mathematical formulation using stochastic gradient descent (SGD).

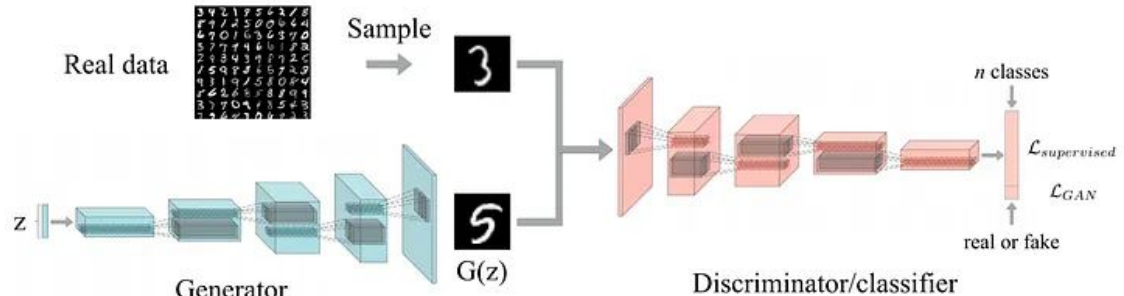
While Vanilla GANs serve as the foundation for more advanced GAN models, they often struggle with issues like mode collapse and unstable training.



Deep Convolutional GANs (DCGANs)

- Deep Convolutional GANs (DCGANs) are among the most popular and widely used types of GANs, particularly for image generation.
- What Makes DCGAN Special?
 - Uses Convolutional Neural Networks (CNNs) instead of simple multi-layer perceptrons (MLPs).
 - Fully connected layers are removed, allowing for better spatial understanding of images.

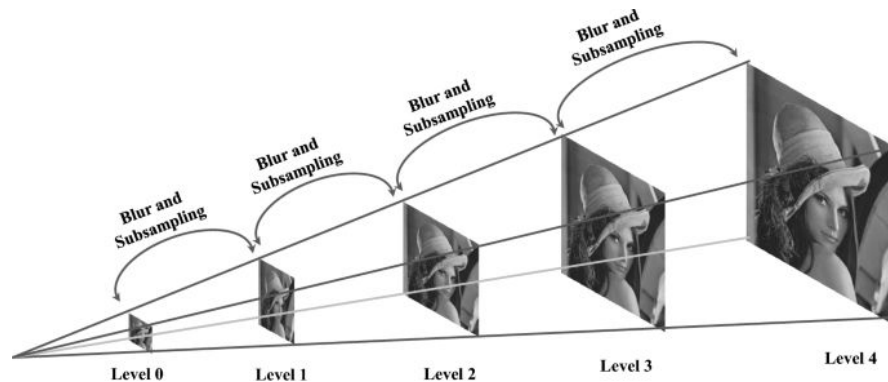
DCGANs have been highly successful in **generating high-quality images**, making them a go-to choice for deep learning researchers.



Laplacian Pyramid GAN (LAPGAN)

- Laplacian Pyramid GAN (LAPGAN) is designed to generate ultra-high-quality images by leveraging a multi-resolution approach.
- Working of LAPGAN:
 - Uses multiple generator-discriminator pairs at different levels of the Laplacian pyramid.
 - Images are first downsampled at each layer of the pyramid and upscaled again using Conditional GANs (CGANs).
 - This process allows the image to gradually refine details, reducing noise and improving clarity.

Due to its ability to generate highly detailed images, LAPGAN is considered a superior approach for **photorealistic image generation**.



Super-Resolution GAN (SRGAN)

- Super-Resolution GAN (SRGAN) is specifically designed to increase the resolution of low-quality images while preserving details.
- Working of SRGAN:
 - Uses a deep neural network combined with an adversarial loss function.
 - Enhances low-resolution images by adding finer details, making them appear sharper and more realistic.
 - Helps reduce common image upscaling errors, such as blurriness and pixelation.



LR image

SRGAN
→



4x HR image

A series of white, stylized lines on the left side of the slide, resembling a circuit board or a series of parallel paths that curve and branch out.

Applications of GANs

Applications

- **Image Synthesis & Generation:** GANs generate realistic images, avatars, and high-resolution visuals by learning patterns from training data. They are widely used in art, gaming, and AI-driven design.
- **Image-to-Image Translation:** GANs can transform images between domains while preserving key features. Examples include converting day images to night, sketches to realistic images, or changing artistic styles.
- **Text-to-Image Synthesis:** GANs create visuals from textual descriptions, enabling applications in AI-generated art, automated design, and content creation.
- **Data Augmentation:** GANs generate synthetic data to improve machine learning models, making them more robust and generalizable, especially in fields with limited labeled data.
- **High-Resolution Image Enhancement:** GANs upscale low-resolution images, improving clarity for applications like medical imaging, satellite imagery, and video enhancement.

Applications

- **Evaluation Metrics for GANs:** Metrics such as the **Inception Score (IS)** and **Fréchet Inception Distance (FID)** are used to quantitatively evaluate the quality and diversity of generated images.
 - **IS: Quality:** Are the generated images clear and realistic? **Diversity:** Do the generated images cover multiple categories?
 - **FID:** How close the distribution of generated images is to the distribution of real images.
- **Ethical Considerations:** The technology behind GANs raises important ethical issues, including the potential creation of deepfakes, the misuse of generated content, and inherent biases in training data.
- **Emerging Research Trends:** Current research is focused on improving GAN stability and interpretability, integrating GANs with other architectures like Variational Autoencoders (VAEs), and expanding their application to areas beyond image processing such as text and music generation.